Unit 4: Object Oriented Programming with Python

4.1 Class and Object

What is a Class?
A **class** in Python is a blueprint for creating objects. It defines the structure and behavior (data and functions) that the objects created from the class will have.
Syntax:

```
class ClassName:
    # attributes and methods
```

What is an Object?
An **object** is an instance of a class. When a class is defined, only the description of the object is created. To use the class, you create an object from it.

Example: Class and Object:

```
class Student:
    def display(self):
        print("I am a student.")
# Creating an object of the class
s1 = Student()
s1.display()
```

4.2 __init__() Method

What is __init__()?

- The __init__() method is a **special method** in Python classes.
- It is known as the **constructor** method.
- It is **automatically called** when an object of the class is created.
- Its main job is to **initialize** the attributes (variables) of the object.

Syntax:

```
class ClassName:
        def __init__(self, parameters):
                # initialize attributes
```

Example:

```
class Student:
    def init(self, name, age):
            self.name = name
            self.age = age
```

```
    def display(self):
            print(f"Name: {self.name}, Age: {self.age}")

s1 = Student("Alice", 20)
s1.display()
```

- __init__() initializes name and age for each object.
- self.name = name assigns the value to the object's attribute.
- When s1 is created, __init__() runs automatically with "Alice" and 20.

4.3 self-keyword

- self is a **reference to the current object** (instance) of a class.
- It is used inside class methods to **access or modify object attributes** and methods.
- It must be the **first parameter** of any instance method in a class (though you don't pass it explicitly when calling the method).

Example:

```
class Student:
   def __init__(self, name, age):
      self.name = name  # instance variable
      self.age = age

   def show_info(self):
      print(f"Name: {self.name}, Age: {self.age}")

# Create object
s1 = Student("Abishek", 22)
s1.show_info()
```

Why is self Important?

- Distinguishes **instance variables** from **local variables**.
- Allows each object to **maintain its own data**.
- Enables interaction between methods and attributes within the same object.

Why is self Used?

Purpose	**Explanation**
Access instance variables	Allows each object to maintain its own data
Call instance methods	Helps one method access another within the same object
Differentiate instance vs local var	Avoids name conflict between instance variables and local

Dynamic binding	Refers to the calling object even if multiple instances exist

4.4 Inheritance

Inheritance is a fundamental concept in **Object-Oriented Programming (OOP)** that allows one class (called **child class** or **derived class**) to inherit properties and behaviors (methods and attributes) from another class (called **parent class** or **base class**).

This helps in **code reusability** and **maintainability**.

Syntax:

```
class Parent:
    # parent class attributes and methods

class Child(Parent):
    # child class inherits from Parent
```

Example:

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

# Creating an object of Dog
d = Dog()
d.speak()  # Inherited from Animal
d.bark()   # Defined in Dog
```

Comparison of Different Types of Inheritance in Python

Inheritance allows a class (child/derived) to inherit attributes and methods from another class (parent/base). Below is a comparison of the various inheritance types in Python:

Type of Inheritance	**Description**	**Syntax Example**	**Diagram**	**Use Case**
Single Inheritance	One child class inherits from one parent class.	class B(A):	A → B	Simple code reuse from a single base class.

Multiple Inheritance	One child class inherits from multiple parent classes.	class C(A, B):	A + B → C	Combines functionality from multiple sources.
Multilevel Inheritance	A class inherits from a derived class, forming a chain.	class C(B): class B(A):	A → B → C	Creates a hierarchy with deeper specialization.
Hierarchical Inheritance	Multiple child classes inherit from a single parent class.	class B(A): class C(A):	A → B, A → C	Sharing common functionality across different subclasses.
Hybrid Inheritance	A combination of two or more types of inheritance.	Mix of above	Complex graph	Models complex relationships; careful MRO (method resolution order) handling required.

Example for Each types of inheritence:

1. Single Inheritance

```
class A:
    def display(self):
        print("A")

class B(A):
    pass

B().display()  # Output: A
```

2. Multiple Inheritance

```
class A:
    def show(self):
        print("A")

class B:
    def display(self):
        print("B")

class C(A, B):
    pass

obj = C()
```

```
obj.show()      # From A
obj.display()   # From B
```

3. Multilevel Inheritance

```
class A:
    def a_method(self):
        print("A")

class B(A):
    def b_method(self):
        print("B")

class C(B):
    def c_method(self):
        print("C")

obj = C()
obj.a_method()  # A
obj.b_method()  # B
obj.c_method()  # C
```

4. Hierarchical Inheritance

```
class A:
    def show(self):
        print("Parent A")

class B(A):
    pass

class C(A):
    pass

B().show()
C().show()
```

5. Hybrid Inheritance

```
class A:
    def method_a(self):
        print("A")

class B(A):
```

```
    def method_b(self):
        print("B")

class C:
    def method_c(self):
        print("C")

class D(B, C):
    pass

obj = D()
obj.method_a()
obj.method_b()
obj.method_c()
```

4.5 Polymorphism and Data Hiding

Polymorphism
Definition:

Polymorphism means "many forms." In Python, it allows functions/methods to use objects of different classes through a common interface. It refers to the ability of different object types to respond to the same function or method call in a way specific to their class. It helps us write flexible and reusable code. Code becomes more flexible and reusable with common interfaces.

Example: Polymorphism with Classes

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

def animal_sound(animal):
    print(animal.speak())

animal_sound(Dog())   # Output: Woof!
animal_sound(Cat())   # Output: Meow!
```

Data Hiding (Encapsulation)

Definition:

Data Hiding is a core concept of **Encapsulation** in Object-Oriented Programming (OOP). It means **restricting direct access to some components (variables or methods) of an object**, to protect the internal state and ensure controlled interaction.

Why Data Hiding?

- Prevents accidental or unauthorized modification of data.
- Keeps the internal representation hidden from outside interference.
- Provides control over how data is accessed or modified through *getter* and *setter* methods.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.__age = age   # private attribute

    def get_age(self):
        return self.__age

    def set_age(self, age):
        if age > 0:
            self.__age = age

obj = Student("Abishek", 21)
print(obj.name)        # Public access → OK
# print(obj.__age)     # ❌ Error: can't access private attribute
print(obj.get_age())   # ✅ Access via method: 21
```